Before Microservices

- Monolithic architecture: multiple components are combined in single large app.
- Single Code Base
- Deployed in single bundle
- Change in one service then whole app is redeployed
- Building problem: developers has to communicate
- Problem in scale
- Cumbersome over time



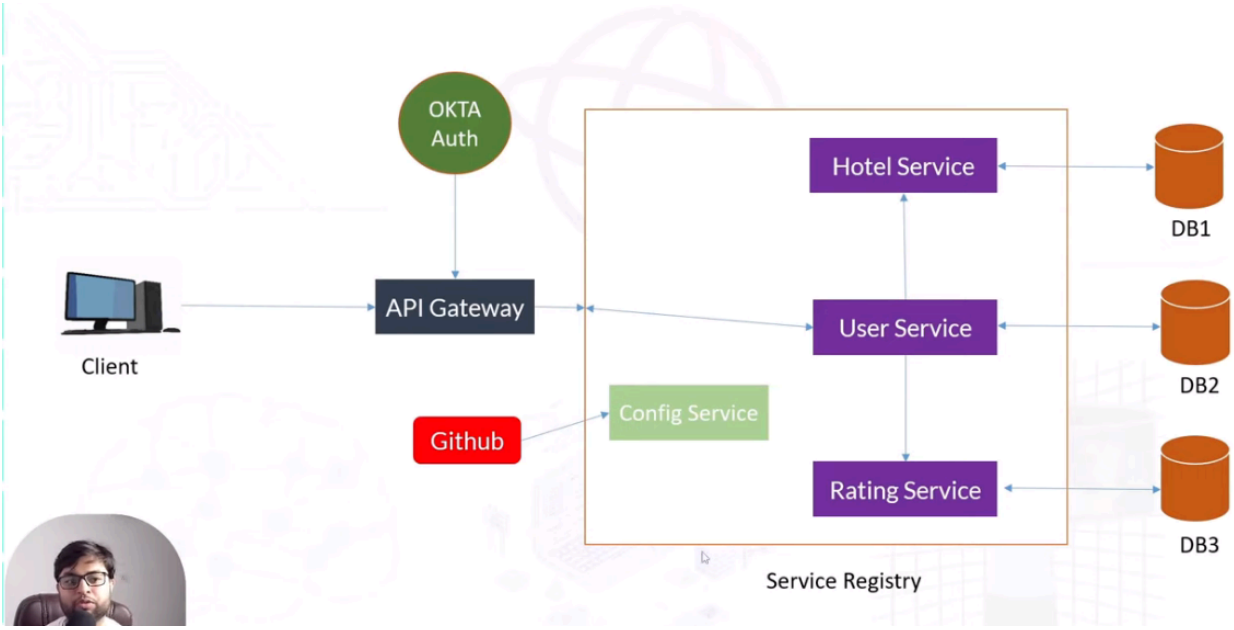
Application using Monolithic

Microservices

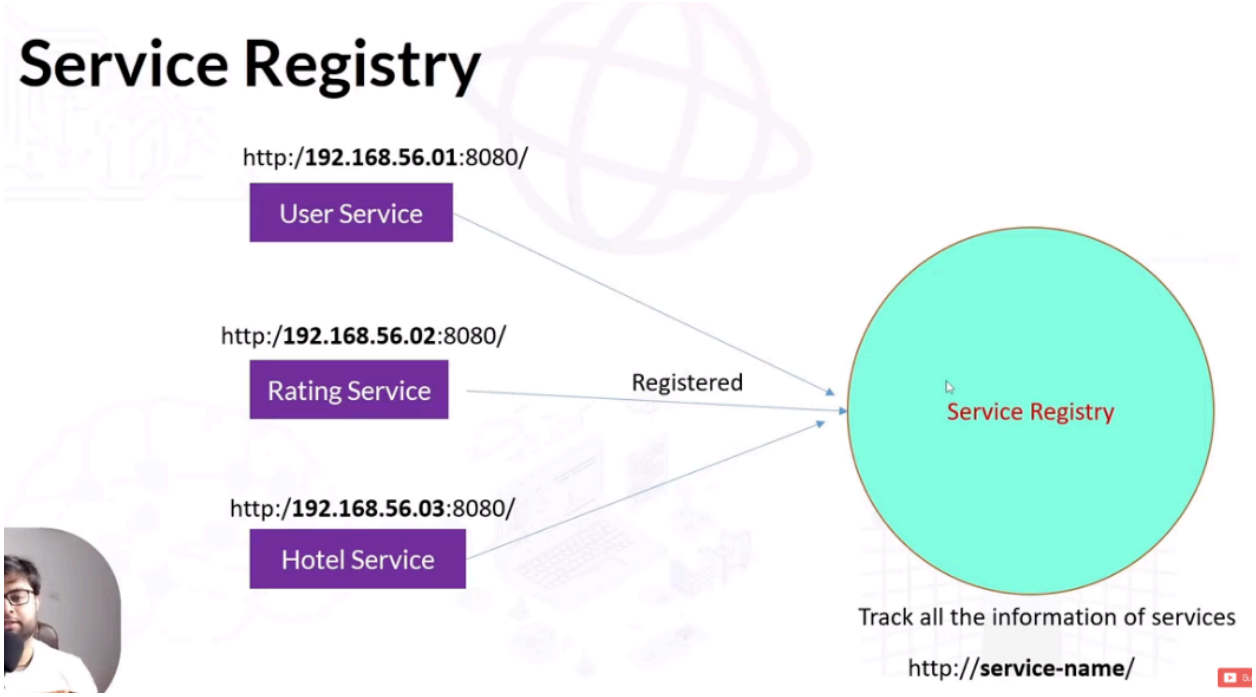
- Large apps are divide into small parts
- Different codebase
- Each module managed independently
- Different tech stack
- Handling microservices is complex

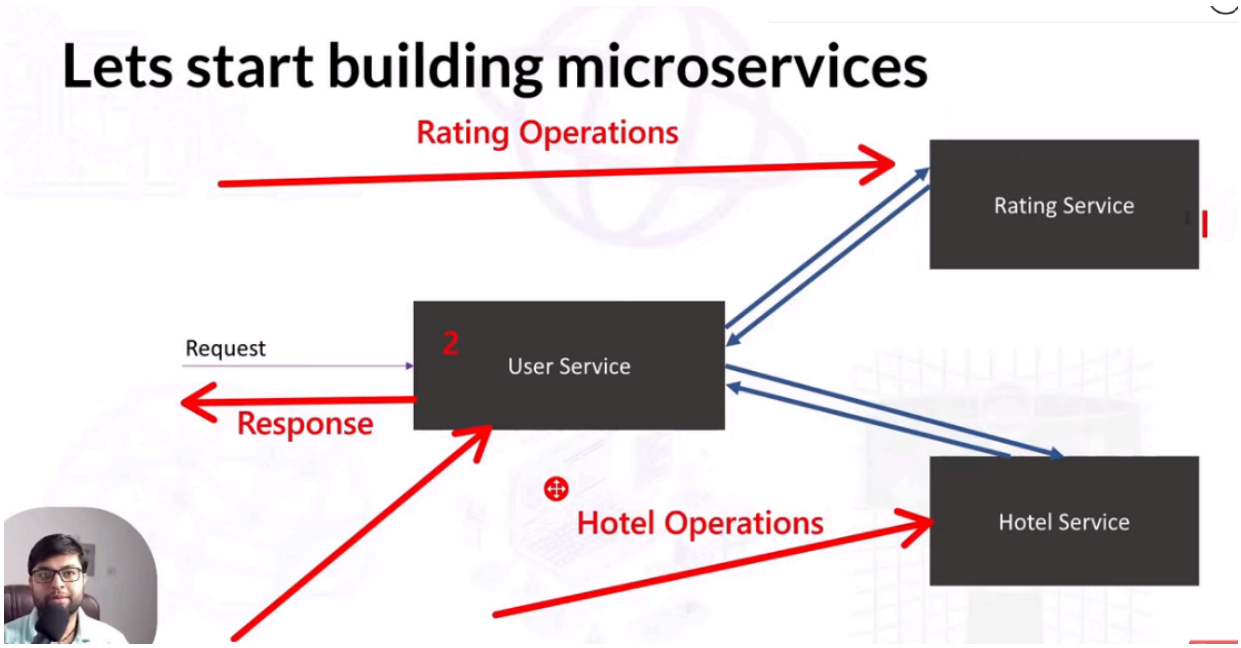


Microservices



->API Gateway->urls for all the services
->OKTA Auth->3rd party authentication to authenticate the user
->Config Service->the configurations which are common in all the services are defined here .
->Service Registry.->We register all the services here .Provide centralized registry where our all services gets stored and we can then call them by their name(no need of port no).If any service is up or down ,its name will be shown here.Basically it track all informations of services.

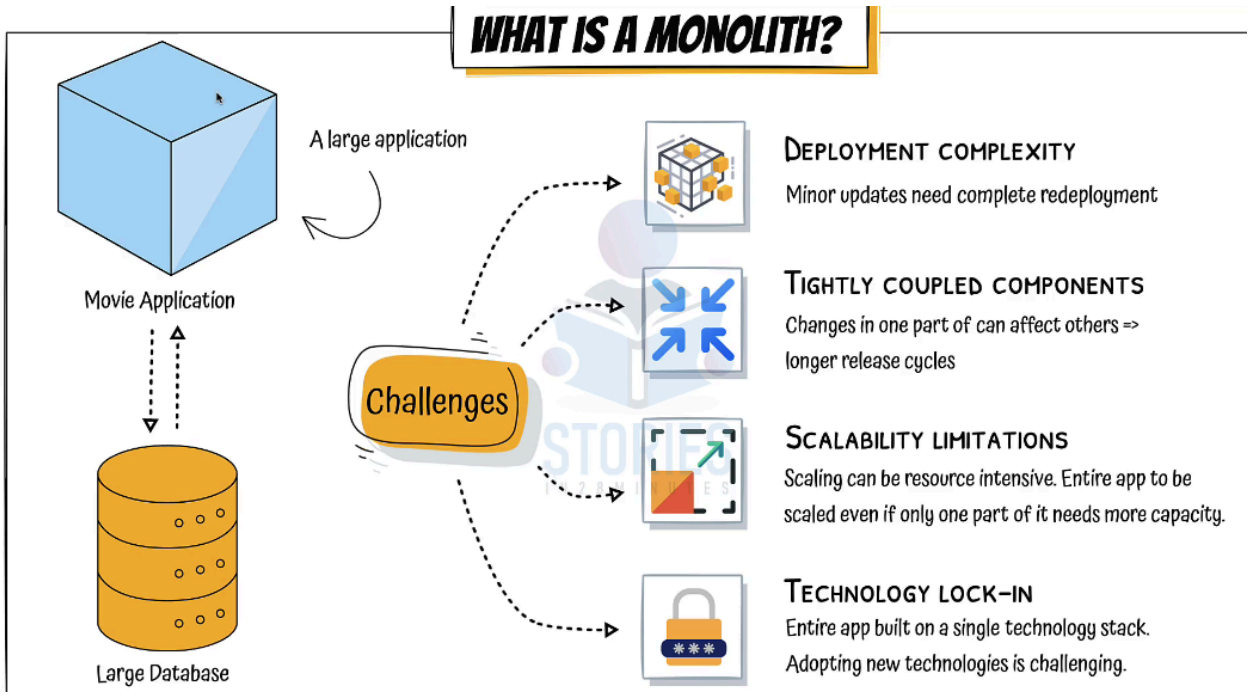


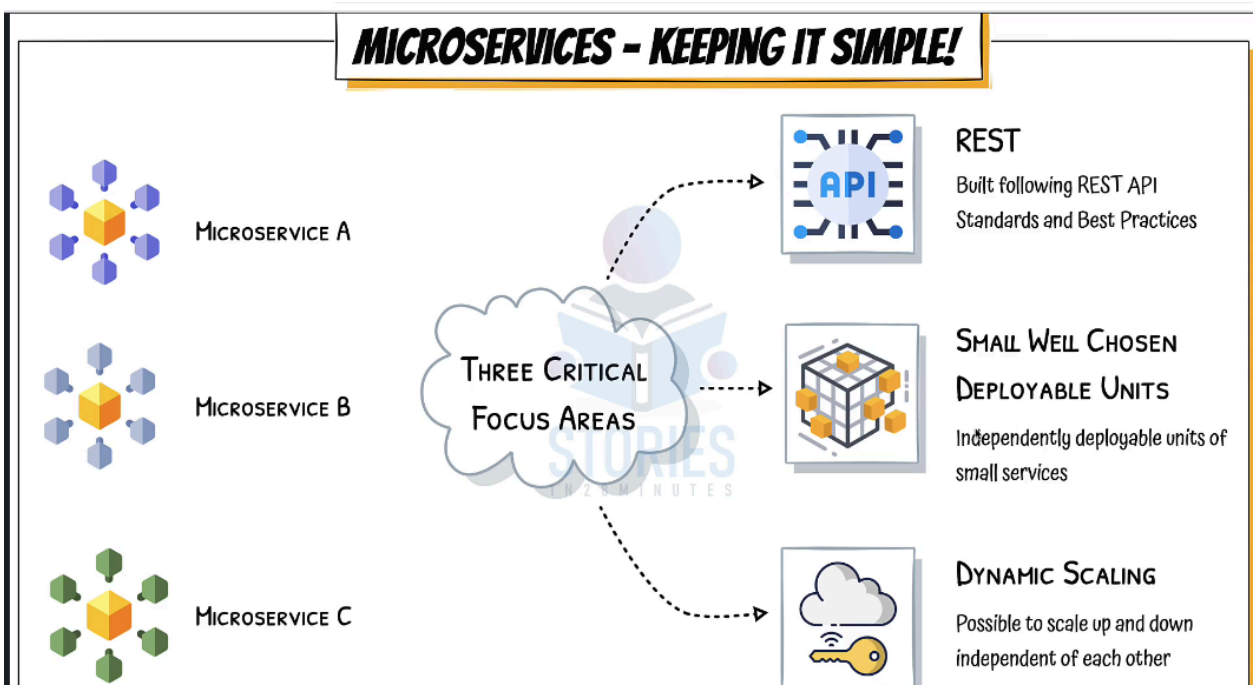


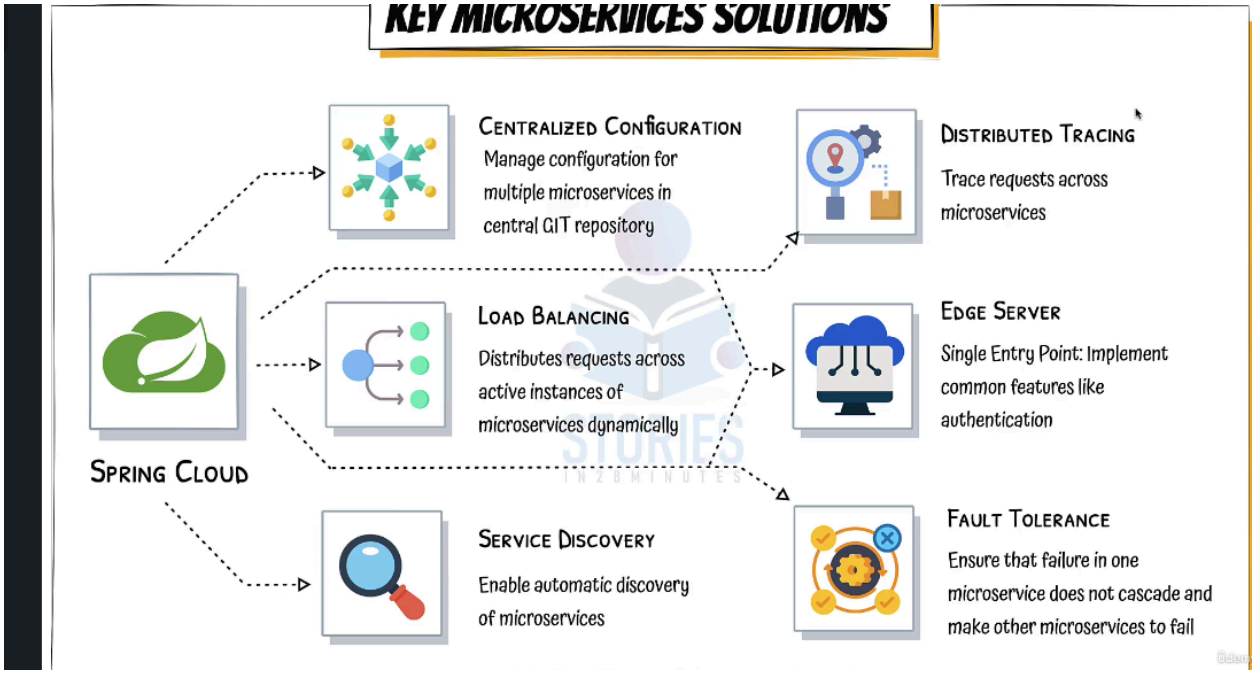
*Important->

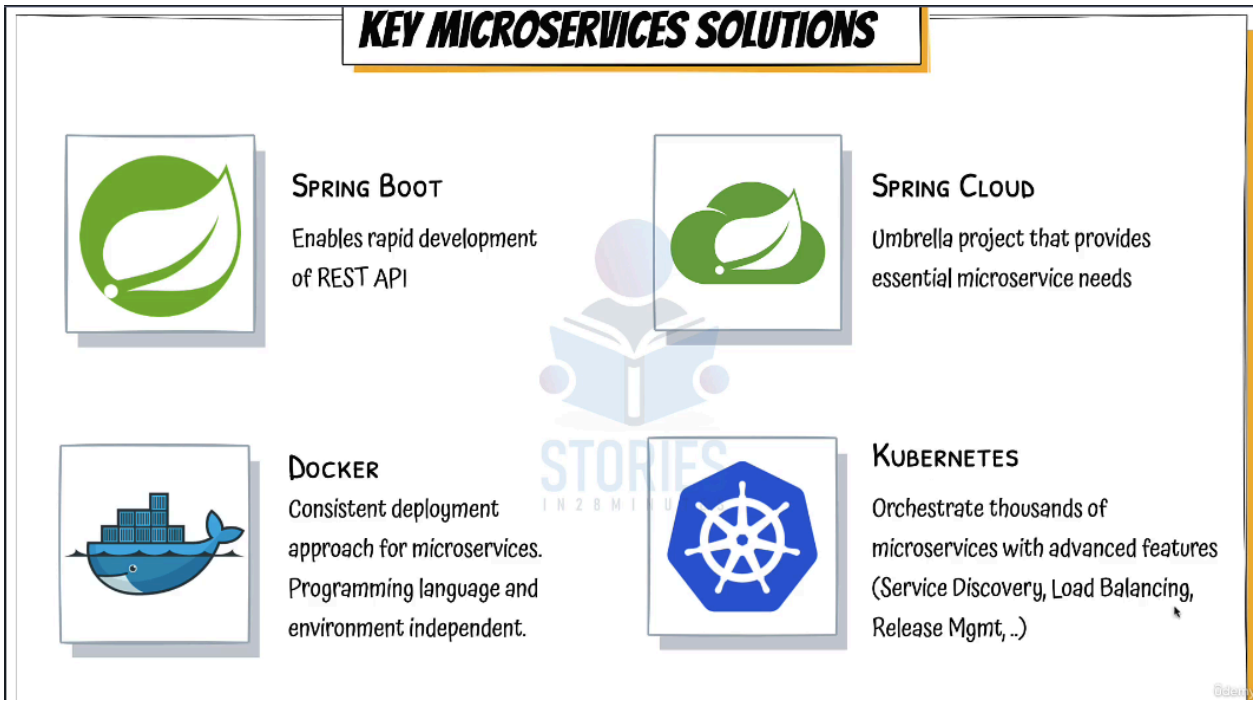
-@Transient indicate that we donot want to store in Db.

-Earlier in the monolithic project, everything is bundled together. If you divide those modules and make them seperated project then that architecture will be called as ""microservices". But instead of calling functionality via method calls, it will be communicated via API. Thats it

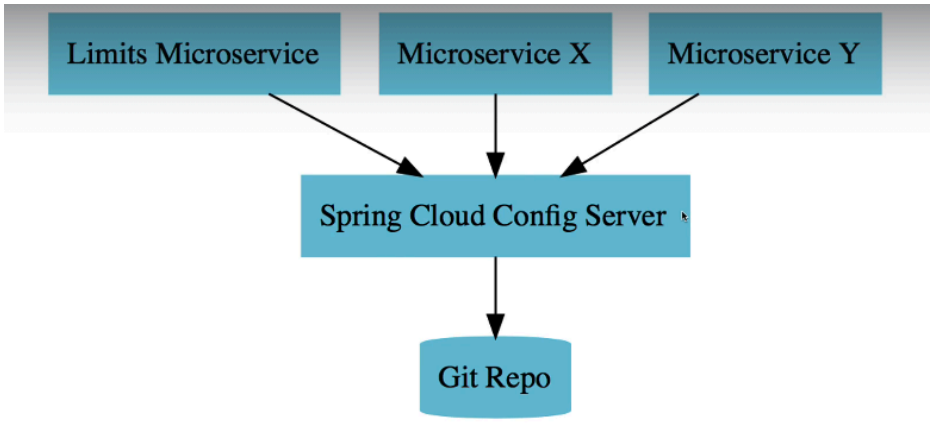

WHAT IS A MONOLITH?
A large application
Movie Application
Large Database
Challenges
DEPLOYMENT COMPLEXITY
Minor updates need complete redeployment
TIGHTLY COUPLED COMPONENTS
Changes in one part of can affect others => longer release cycles
SCALABILITY LIMITATIONS
Scaling can be resource intensive. Entire app to be scaled even if only one part of it needs more capacity.
TECHNOLOGY LOCK-IN
Entire app built on a single technology stack. Adopting new technologies is challenging.


MICROSERVICES - KEEPING IT SIMPLE!
MICROSERVICE A
MICROSERVICE B
MICROSERVICE C
THREE CRITICAL FOCUS AREAS
API
REST
Built following REST API Standards and Best Practices
SMALL WELL CHOSEN DEPLOYABLE UNITS
Independently deployable units of small services
DYNAMIC SCALING
Possible to scale up and down independent of each other





Need for Centralized Configurations->



Config Server

1)Set the Limits Microservice

Limits microservice will talk to spring cloud config server and then Spring cloud config server will fetch configuration from Git repo.

->@ConfigurationProperties("limits-service") → to get the values of the variables defined in application.properties.

Enhanced the limit service to get the properties from a property file→using @Configu…..

2)Setting up the Spring Cloud Config Server—> ADD dependency -Config Server Standarized port-8888

-Values in application.properties has less priorities then values in git repo.

-If there are multiple environments for the Limits MicroService- dev envir ,QA stage production environ .How do I store the seperate configur for limit service for these environments,and how do I make use of it from Spring Cloud Config Server.

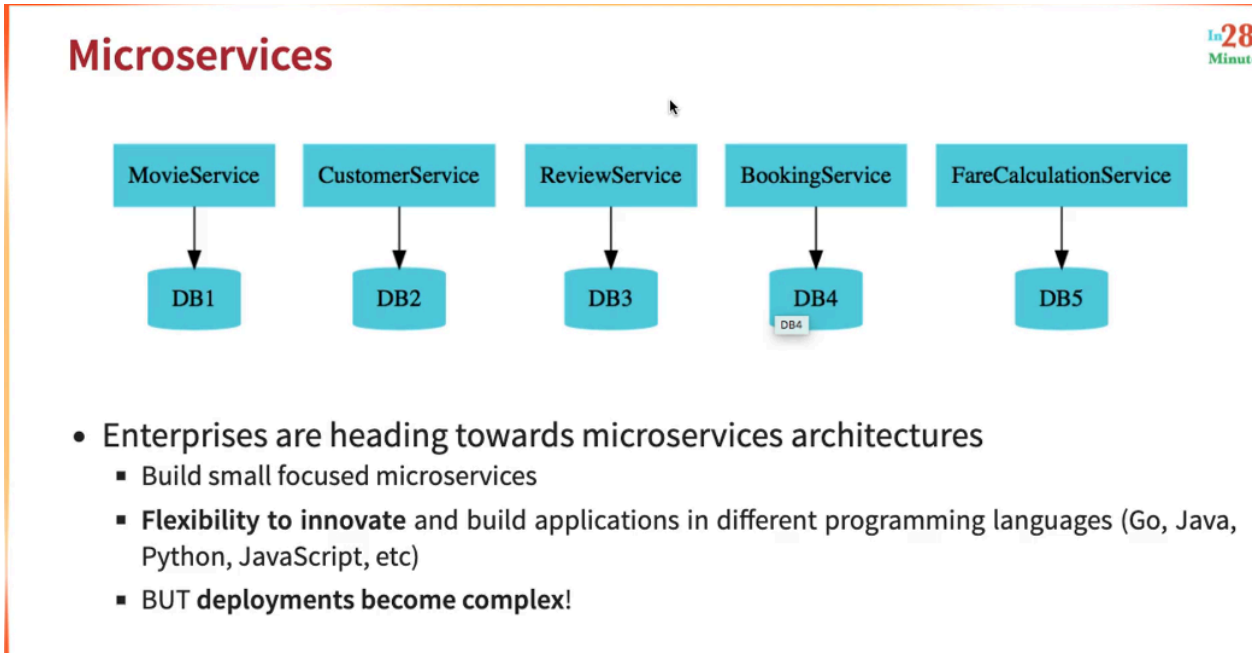
->Spring Boot Actuator?

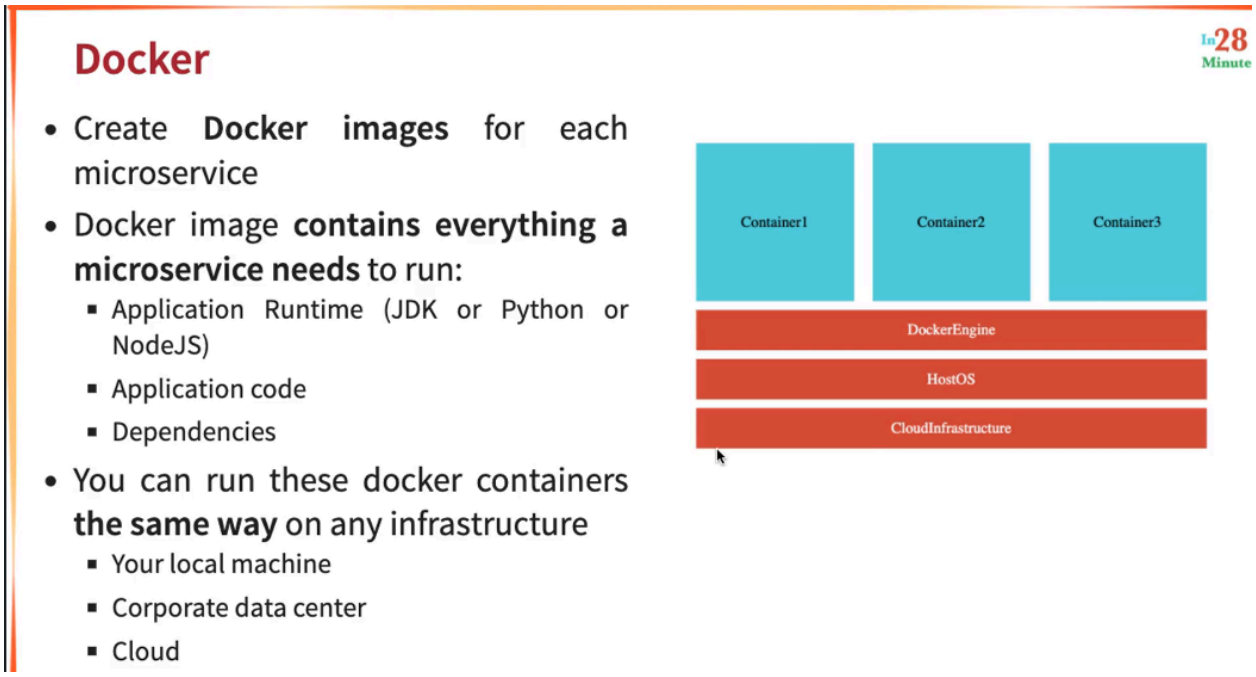
```
spring.application.name=currency-exchange
server.port=8000
spring.config.import=optional:configserver:http://localhost:8888

```

3 - Connecting to the spring cloud config server.

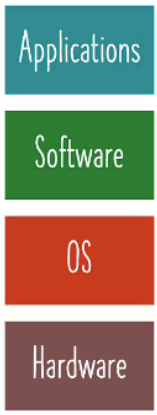
-> Feign- reduce the amount of code we need to write to communicate between the 2 diff microservices.





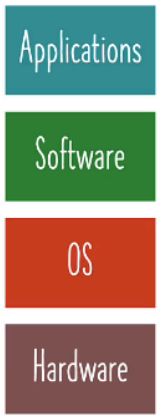
How does Traditional Deployment work?

- Deployment process **described in a document**
- Operations team **follows steps** to:
  - Setup Hardware
  - Setup OS (Linux, Windows, Mac, ...)
  - Install Software (Java, Python, NodeJs, ...)
  - Setup Application Dependencies
  - Install Application
- **Manual approach**:
  - Takes a lot of time
  - High chance of making mistakes



Understanding Deployment Process with Docker

- **Simplified** Deployment Process:
  - OS doesn't matter
  - Programming Language does not matter
  - Hardware does not matter
- **01:** Developer creates a Docker Image
- **02:** Operations run the Docker Image
  - Using a very simple command
- **Takeaway**: Once you have a Docker Image, irrespective of what the docker image contains, you run it the same way!
  - Make your operations team happy



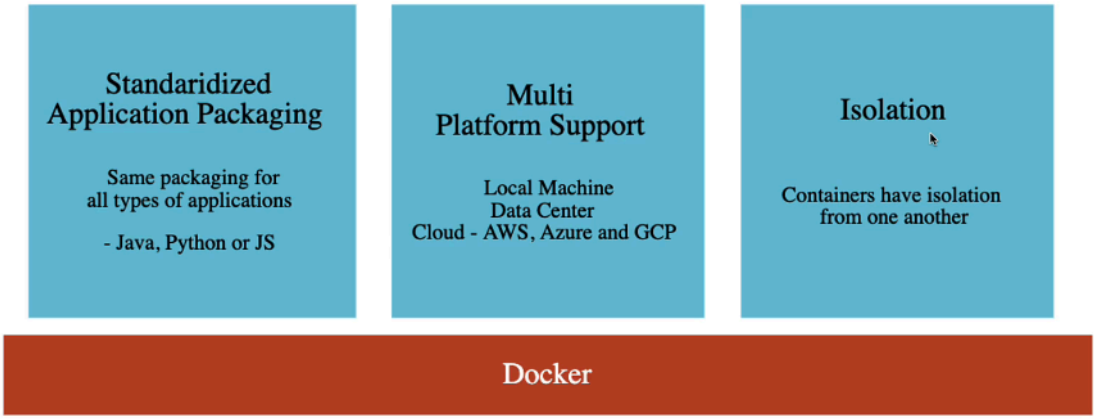
How does Docker Make it Easy?

In 28 Minutes

- Docker image **has everything you need to run your application**:
  - Operating System
  - Application Runtime (JDK or Python or NodeJS)
  - Application code and dependencies
- You can run a Docker container **the same way** everywhere:
  - Your local machine
  - Corporate data center
  - Cloud



Why is Docker Popular?



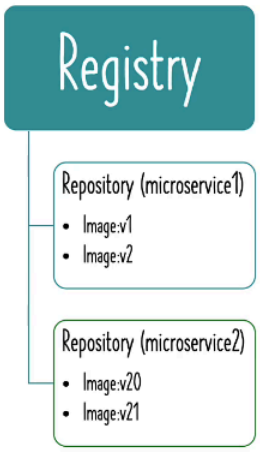
What's happening in the Background?

```
docker container run -d -p 5000:5000 in28min/hello-world-nodejs:0.0.1.RELEASE
```

- **Docker image** is downloaded from Docker Registry (Default: Docker Hub)
  - *https://hub.docker.com/r/in28min/hello-world-nodejs*
  - **Image** is a set of bytes
  - **Container**: Running Image
  - **in28min/hello-world-nodejs**: Repository Name
  - **0.0.1.RELEASE**: Tag (or version)
  - **-p hostPort:containerPort**: Maps internal docker port (container port) to a port on the host (host port)
    - By default, Docker uses its own internal network called bridge network
    - We are mapping a host port so that users can access your application
  - **-d**: Detatched Mode (Don't tie up the terminal)

Understanding Docker Terminology

- **Docker Image**: A package representing specific version of your application (or software)
  - Contains everything your app needs
    - OS, software, code, dependencies
- **Docker Registry**: A place to store your docker images
- **Docker Hub**: A registry to host Docker images
- **Docker Repository**: Docker images for a specific app (tags are used to differentiate different images)
- **Docker Container**: Runtime instance of a docker image
- **Dockerfile**: File with instructions to create a Docker image



Dockerfile - 1 - Creating Docker Images

```
FROM openjdk:18.0-slim
COPY target/*.jar app.jar
EXPOSE 5000
ENTRYPOINT ["java","-jar","/app.jar"]
```

- Dockerfile contains instruction to create Docker images
  - **FROM** - Sets a base image
  - **COPY** - Copies new files or directories into image
  - **EXPOSE** - Informs Docker about the port that the container listens on at runtime
  - **ENTRYPOINT** - Configure a command that will be run at container launch

Dockerfile - 2 - Build Jar File - Multi Stage

```
FROM maven:3.8.6-openjdk-18-slim AS build
WORKDIR /home/app
COPY . /home/app
RUN mvn -f /home/app/pom.xml clean package

FROM openjdk:18.0-slim
EXPOSE 5000
COPY --from=build /home/app/target/*.jar app.jar
ENTRYPOINT [ "sh", "-c", "java -jar /app.jar" ]
```

- Let build the jar file as part of creation of Docker Image
- Your build does NOT make use of anything built on your local machine

Building .jar file from maven and then copying it in app.jar.

Improve build time of docker images->
Layering!

Dockerfile - 3 - Improve Layer Caching

```
FROM maven:3.8.6-openjdk-18-slim AS build
WORKDIR /home/app

COPY ./pom.xml /home/app/pom.xml
COPY ./src/main/java/com/example/demodocker/DemoDockerApplication.java /
    /home/app/src/main/java/com/example/demodocker/DemoDockerApplication.java

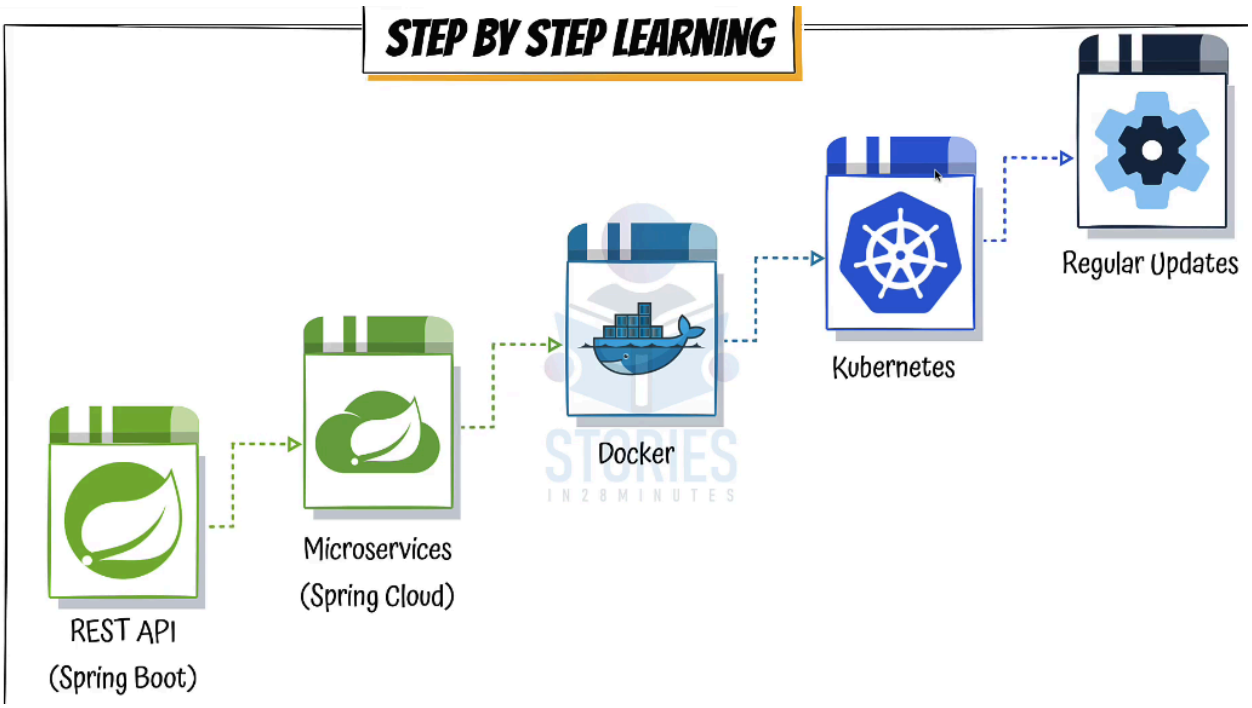
RUN mvn -f /home/app/pom.xml clean package

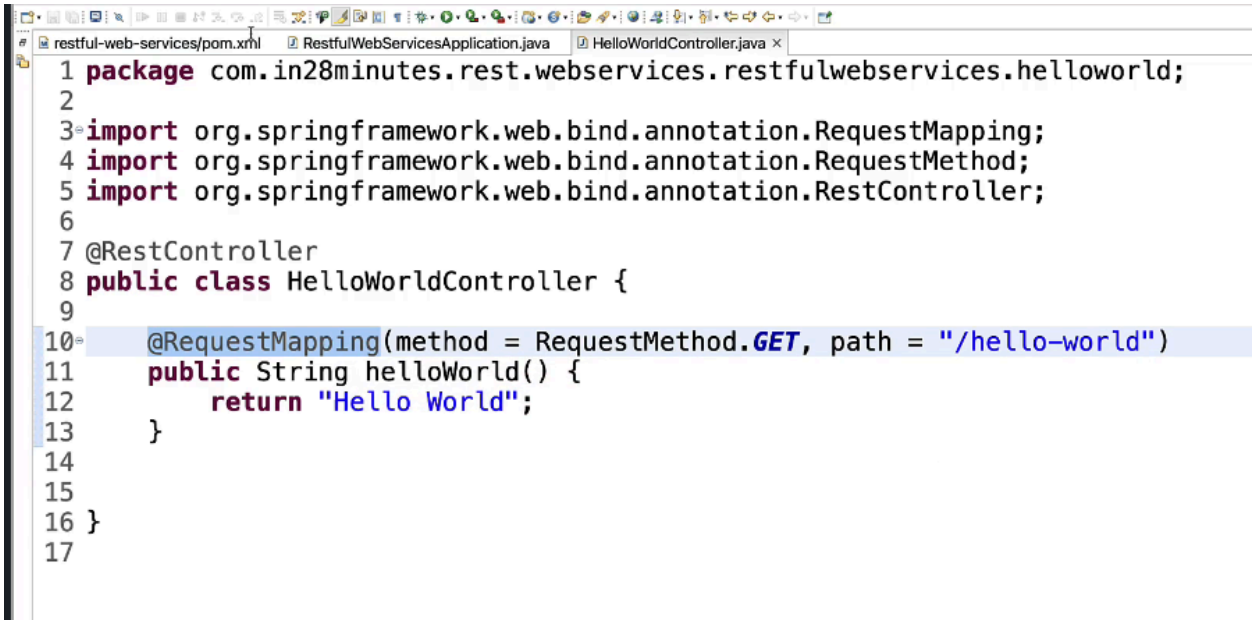
COPY . /home/app
RUN mvn -f /home/app/pom.xml clean package

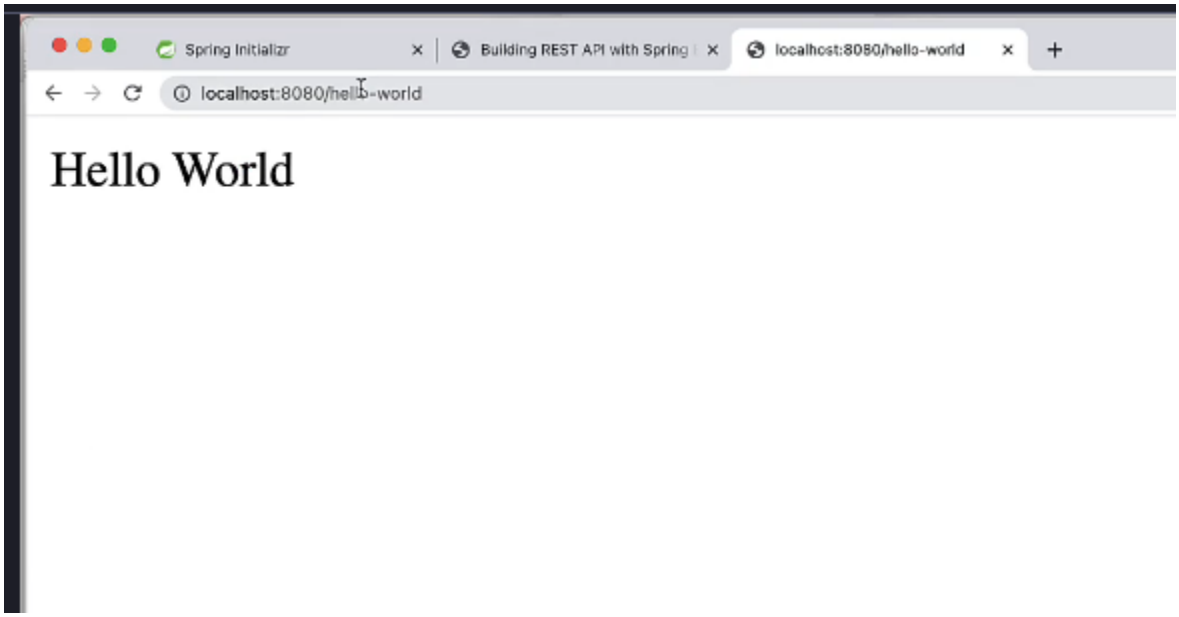
FROM openjdk:18.0-slim
EXPOSE 5000
COPY --from=build /home/app/target/*.jar app.jar
ENTRYPOINT [ "sh", "-c", "java -jar /app.jar" ]
```

- Docker caches every layer and tries to reuse it
- Let's make use of this feature to make our build efficient




STEP BY STEP LEARNING
REST API
(Spring Boot)
Microservices
(Spring Cloud)
Docker
Kubernetes
Regular Updates


restful-web-services/pom.xml
RestfulWebServicesApplication.java
HelloWorldController.java
1 package com.in28minutes.rest.webservices.restfulwebservices.helloworld;
2
3 import org.springframework.web.bind.annotation.RequestMapping;
4 import org.springframework.web.bind.annotation.RequestMethod;
5 import org.springframework.web.bind.annotation.RestController;
6
7 @RestController
8 public class HelloWorldController {
9
10 @RequestMapping(method = RequestMethod.GET, path = "/hello-world")
11 public String helloWorld() {
12 return "Hello World";
13 }
14
15
16 }
17


Spring Initializr
Building REST API with Spring
localhost:8080/hello-world
localhost:8080/hello-world
Hello World

```java
@RestController
public class HelloWorldController {

    @GetMapping(path = "/hello-world")
    public String helloWorld() {
        return "Hello World";
    }


}
```

better to use @GetMapping.
It is returning string.
I want to return JSON-> ??

Create a bean . Return the instance of class->object (in json format)

restful-web-services/pom.xml | RestfulWebServicesApplication.java | HelloWorldController.java | HelloWorldBean.java

```java
package com.in28minutes.rest.webservices.restfulwebservices.helloworld

public class HelloWorldBean {

    private String message;

    public HelloWorldBean(String message) {
        this.message = message;
    }

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }

    @Override
    public String toString() {
        return "HelloWorldBean [message=" + message + "]";
    }
}
```

```
restful-web-services/pom.xml  RestfulWebServicesApplication.java  HelloWorldController.java ×
 1 package com.in28minutes.rest.webservices.restfulwebservices.helloworld;
 2
 3 import org.springframework.web.bind.annotation.GetMapping;
 4 import org.springframework.web.bind.annotation.RestController;
 5
 6 @RestController
 7 public class HelloWorldController {
 8
 9     @GetMapping(path = "/hello-world")
10     public String helloWorld() {
11         return "Hello World";
12     }
13
14     @GetMapping(path = "/hello-world-bean")
15     public HelloWorldBean helloWorldBean() {
16         return new HelloWorldBean("Hello World");
17     }
18
19
20 }
21
```

What is happening in Background??

1) When a request is made ,it is first sent to dispatcherServlet,irrespective of the url I am making use of .Because dispatcherServlet is mapped to the root url.[/]

2) It then map to the the right url .
3) How java object converting in JSON?

```
restful-web-services/pom.x  RestfulWebServicesApplicati  HelloWorldController.java  applicatior
1 logging.level.org.springframework=debug
```

@PathVariable-> giving any parameter in url (value)

```
// GET /users
@GetMapping("/users/{id}")
public User retrieveUser(@PathVariable int id) {
    return service.findOne(id);
}
```

In controller-> we call rest apis..
In service-> the logic is being written.

ResponseEntity->class present in spring boot-> to return response status.

```
return ResponseEntity.created(null).build();
```

location-> to send the url to user so that on clicking it they can directly redirected there.

```
@PostMapping("/users")
public ResponseEntity<User> createUser(@RequestBody User user) {
    User savedUser = service.save(user);
    // /users/4 => /users ,   user.getID
    URI location = ServletUriComponentsBuilder.fromCurrentRequest()
                    .path("/{id}")
                    .buildAndExpand(savedUser.getId())
                    .toUri();
    return ResponseEntity.created(location).build();
}
```

->@Response Status->

Create a class for userNoFoundException extending a particular extension interface present.
@ResponseStatus(code=HttpStatus.NOT_FOUND)

-Making our own Custom Exception.

```
@ControllerAdvice
public class CustomizedResponseEntityExceptionHandler extends ResponseEntityExceptionHanc

    @ExceptionHandler(Exception.class)
    public final ResponseEntity<Object> handleAllExceptions(Exception ex, WebRequest requ
        ErrorDetails errorDetails = new ErrorDetails(LocalDate.now(),
                ex.getMessage(), request.getDescription(false));

        return new ResponseEntity(errorDetails, HttpStatus.INTERNAL_SERVER_ERROR);

    }
```

Here errorDetails is a class, in which we define the structure in which we want to show our exception.(response status,and structure is customised)

Now validating REST APIs ->

If in user entity ,if any of its attribute defined is not provided, eg -email then not accept the post request.

->add dependency-> starter-validation
@Valid

```
@Override
protected ResponseEntity<Object> handleMethodArgumentNotValid(
        MethodArgumentNotValidException ex, HttpHeaders headers, HttpStatusCode statu

    ErrorDetails errorDetails = new ErrorDetails(LocalDateTime.now(),
            "Total Errors:" + ex.getErrorCount() + " First Error:" + ex.getFieldErro


    return new ResponseEntity(errorDetails, HttpStatus.BAD_REQUEST);
}
```

Tell the error to the user-> like if userName missing....

```
@PostMapping("/users")
public ResponseEntity<User> createUser(@Valid @RequestBody User user) {

    User savedUser = service.save(user);

    URI location = ServletUriComponentsBuilder.fromCurrentRequest()
                    .path("/{id}")
                    .buildAndExpand(savedUser.getId())
                    .toUri();

    return ResponseEntity.created(location).build();
}
```

Auto Generation of Swagger Documentation and Open AI->

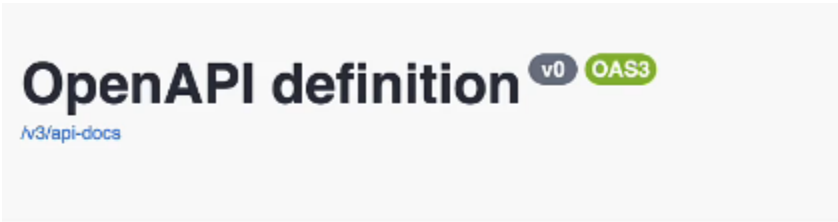
dependency->

`springdoc-openapi` java library helps to automate the generation of API documentation for spring boot projects.

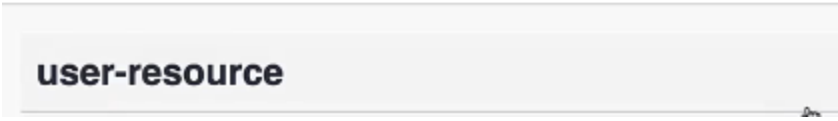
```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.7.0</version>
</dependency>
```

Then

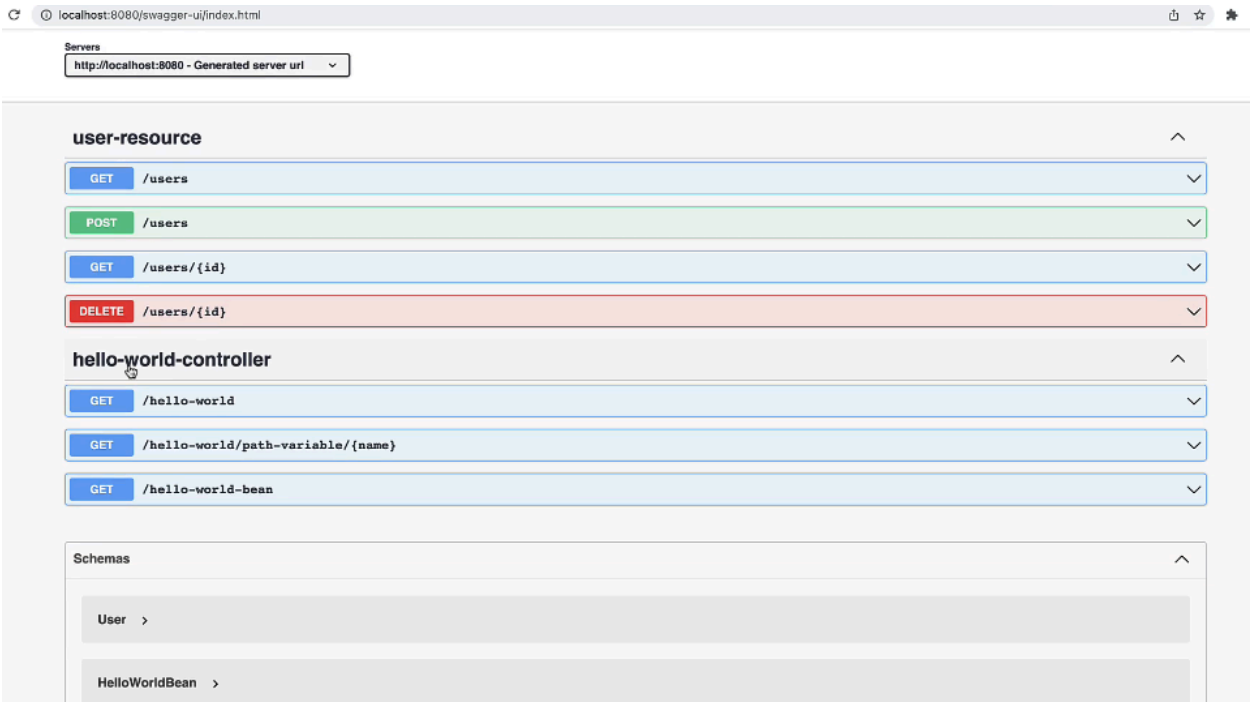
http://localhost:8080/swagger-ui.html







OpenApi provide the definition for our REST APIs and Swagger UI help us to visualize it and play around with.



<u>Content Negotiation</u> -> For a same source,we can have different representations.In CN, the consumer and service provide negotiate with each other.Consumer tell what kind of repres.. It want xml response,json response ,this is done by accept header.

Accept header-> tell in which we want our response we just getting it is enabled by adding just a dependency.

```
<dependency>
    <groupId>com.fasterxml.jackson.dataformat</groupId>
    <artifactId>jackson-dataformat-xml</artifactId>
</dependency>
```



Internationalization(i18n) of REST APIs->

MessageSource-> is a interface,that implement i18n

```
private MessageSource messageSource;

public HelloWorldController(MessageSource messageSource) {
    this.messageSource = messageSource;
}
```

Versioning REST API-> adding breakage

```
    return new PersonV1("Bob Charlie");
}

@GetMapping("/v2/person")
public PersonV2 getSecondVersionOfPerson() {
    return new PersonV2(new Name("Bob", "Charlie"));
}

@GetMapping(path = "/person", params = "version=1")
public PersonV1 getFirstVersionOfPersonRequestParameter() {
    return new PersonV1("Bob Charlie");
}

@GetMapping(path = "/person", params = "version=2")
public PersonV2 getSecondVersionOfPersonRequestParameter() {
    return new PersonV2(new Name("Bob", "Charlie"));
}

@GetMapping(path = "/person/header", headers = "X-API-VERSION=1")
public PersonV1 getFirstVersionOfPersonRequestHeader() {
    return new PersonV1("Bob Charlie");
}

}
```

```java
private Integer id;

@Size(min=2, message = "Name should have atleast 2 characters")
@JsonProperty("user_name")
private String name;

@Past(message = "Birth Date should be in the past")
@JsonProperty("user_name")
private LocalDate birthDate;
```

```
{
    "id": 1,
    "birthDate": "1992-08-16",
    "user_name": "Adam"
},
{
    "id": 2,
    "birthDate": "1997-08-16",
    "user_name": "Eve"
},
```

```
{
    "id": 1,
    "user_name": "Adam",
    "birth_date": "1992-08-16"
},
{
    "id": 2,
    "user_name": "Eve",
    "birth_date": "1997-08-16"
},
```

Customizing REST API Responses - Filtering and more..

- **Serialization**: Convert object to stream (example: JSON)
  - Most popular JSON Serialization in Java: Jackson
- How about customizing the REST API response returned by Jackson framework?
- **1:** Customize field names in response
  - @JSONProperty
- **2:** Return only selected fields
  - **Filtering**
  - Example: Filter out Passwords
  - **Two types**:
    - **Static Filtering**: Same filtering for a bean across different REST API
      - @JsonIgnoreProperties, @JsonIgnore
    - **Dynamic Filtering**: Customize filtering for a bean for specific REST API
      - @JsonFilter with FilterProvider



Static Filtering->Same filtering across all rest apis

```
import com.fasterxml.jackson.annotation.JsonIgnore;
import com.fasterxml.jackson.annotation.JsonIgnoreProperties;

@JsonIgnoreProperties("field1")
public class SomeBean {
    private String field1;

    @JsonIgnore
    private String field2;

    private String field3;

    public SomeBean(String field1, String field2, String field3) {
        super();
        this.field1 = field1;
        this.field2 = field2;
        this.field3 = field3;
    }
```

```java
@RestController
public class FilteringController {

    @GetMapping("/filtering") //field2
    public MappingJacksonValue filtering() {

        SomeBean someBean = new SomeBean("value1","value2", "value3");

        MappingJacksonValue mappingJacksonValue = new MappingJacksonValue(someBean);

        SimpleBeanPropertyFilter filter =
                SimpleBeanPropertyFilter.filterOutAllExcept("field1","field3");

        FilterProvider filters =
                new SimpleFilterProvider().addFilter("SomeBeanFilter", filter );

        mappingJacksonValue.setFilters(filters );


        return mappingJacksonValue;
```

Spring Boot Actuator->

Provides Spring Boots production ready features.
dependency-> spring-boot-starter-actuator

*Untitled 2 × Customizer.class

```
Spring Security

Filter Chains

1) All requests should be authenticated
2) If a request is not authenticated, a web page is shown
3) CSRF -> POST, PUT

Configuration

Bean FilterChain


```

```java
@Configuration
public class SpringSecurityConfiguration {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

//      1) All requests should be authenticated
        http.authorizeHttpRequests(
                auth -> auth.anyRequest().authenticated()
                );
//      2) If a request is not authenticated, a web page is shown
        http.httpBasic(withDefaults());

//      3) CSRF -> POST, PUT
        http.csrf().disable();


        return http.build();
    }

}
```

